

Mahidol University
Faculty of Information
and Communication Technology

User Environment

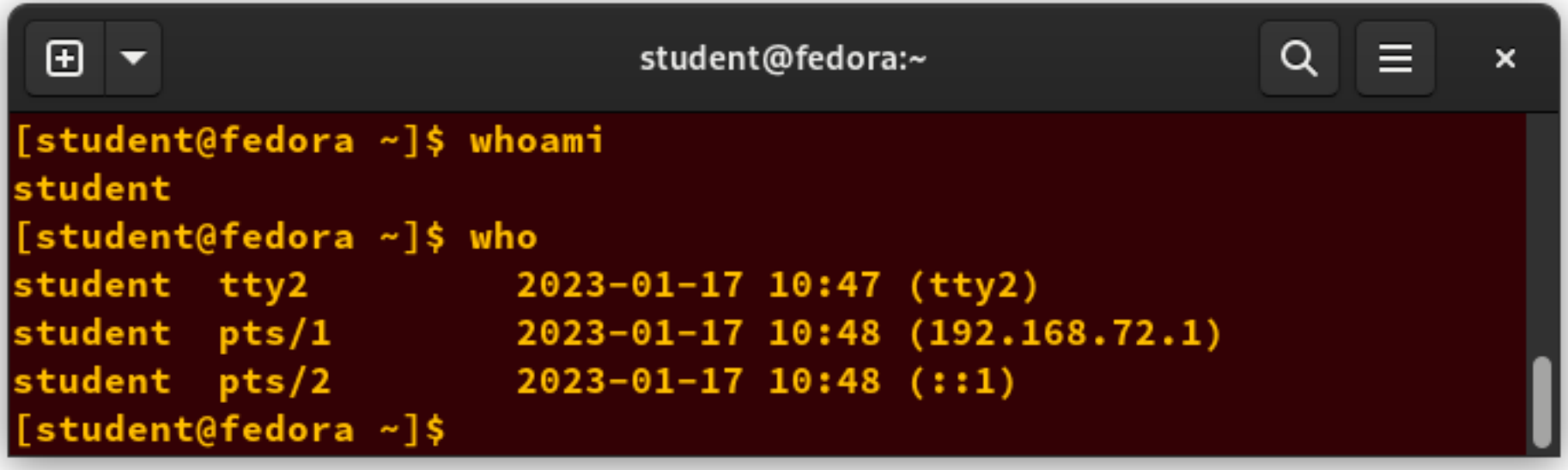
ITCS 254 | Lecture 9

Ittipon (Art) Rassameeroj

Agenda

- Accounts, Users, and Groups
- Environment Variables
- Recalling Previous Commands
- File Permissions

Identifying the Current User



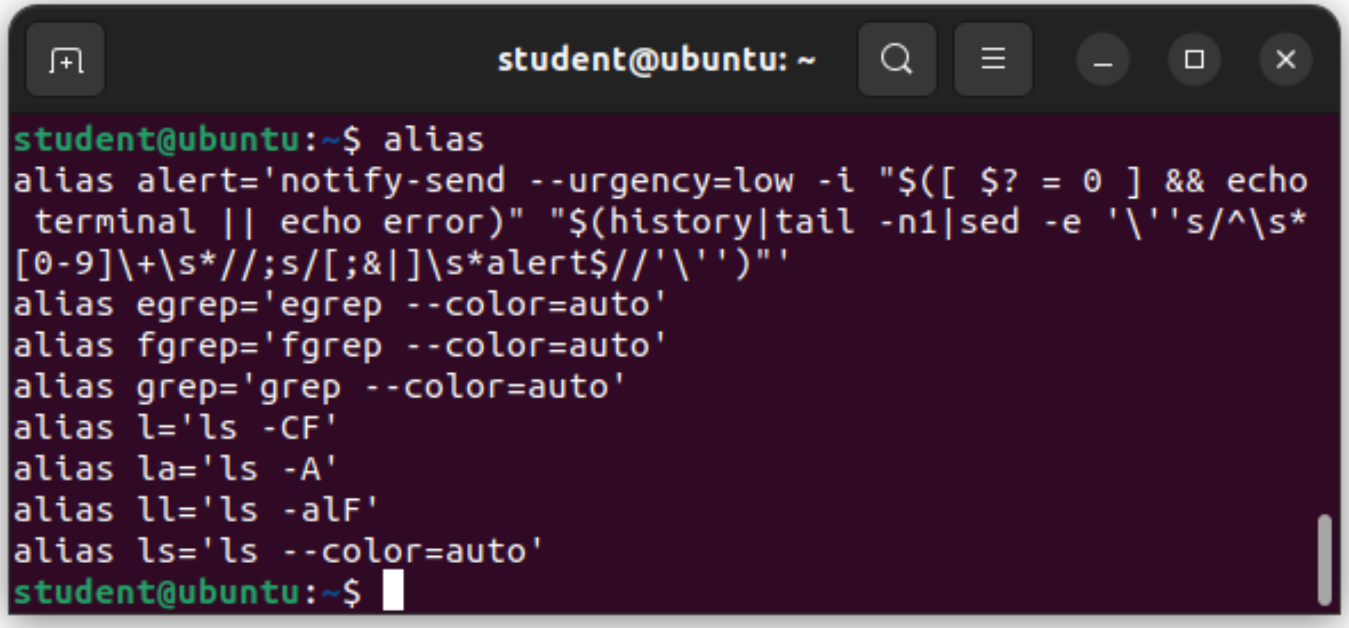
User Startup Files

- In Linux, the command shell program (generally bash) uses one or more startup files to configure the user environment.
- Files in the `/etc` directory define global settings for all users, while initialization files in the user's home directory can include and/or override the global settings.
- The startup files can do anything the user would like to do in every command shell, such as:
  - Customizing the prompt
  - Defining command line shortcuts and aliases
  - Setting the default text editor
  - Setting the path for where to find executable programs

Creating Alias

- Most often, these aliases are placed in your `~/.bashrc` file so they are available to any command shells you create.

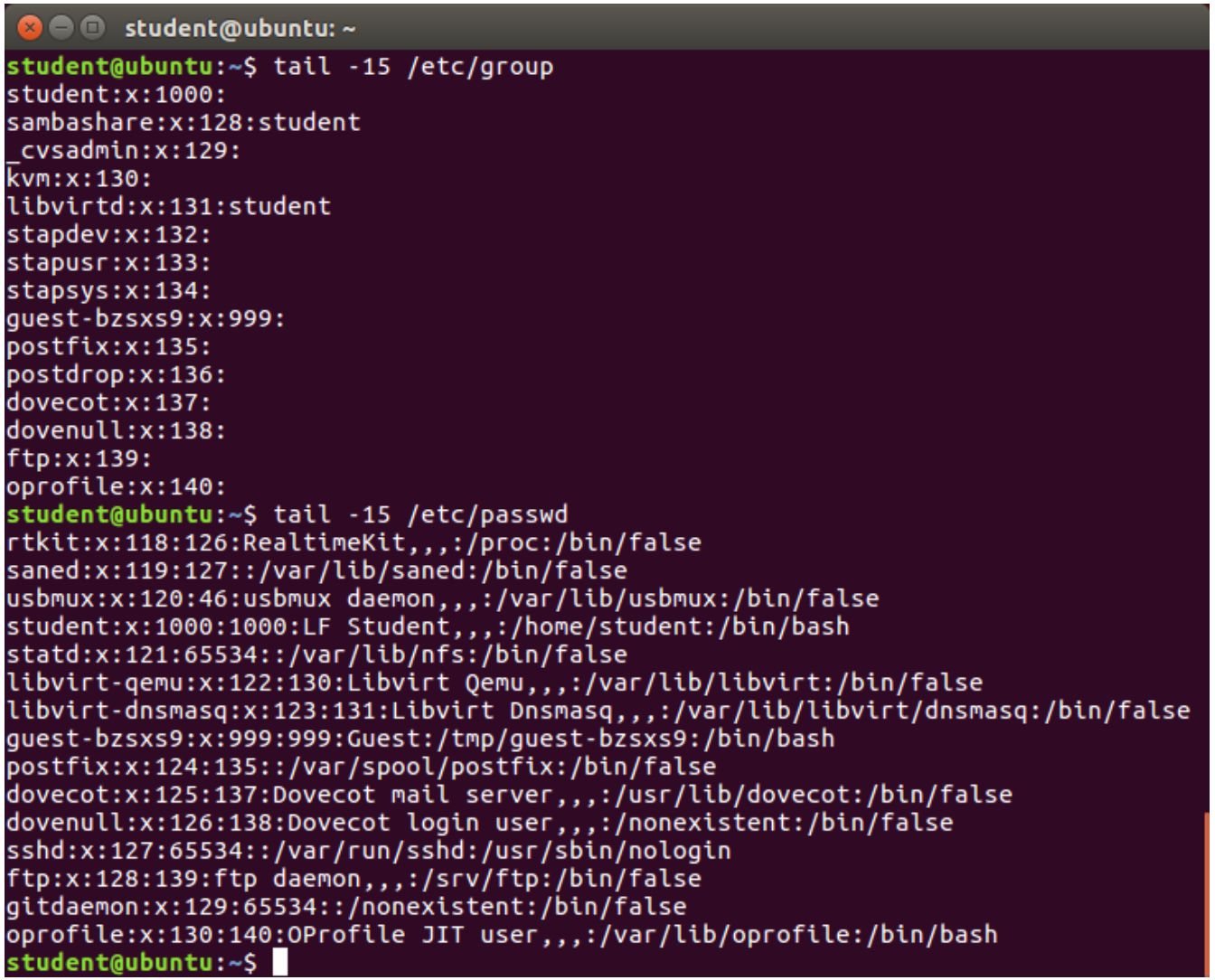
- `unalias` removes an alias.



Basics of Users and Groups

- All Linux users are assigned a unique user ID (`uid`), which is just an integer; normal users start with a `uid` of 1000 or greater.
- Linux uses groups for organizing users. Groups are collections of accounts with certain shared permissions.
- Control of group membership is administered through the `/etc/group` file, which shows the list of groups and their members.
  - By default, every user belongs to a default (primary) group.
- Users also have one or more group IDs (`gid`), including a default one that is the same as the user ID.
  - These numbers are associated with names through the files `/etc/passwd` and `/etc/group`.

Basics of Users and Groups



Adding and Removing Users

```
$ sudo useradd <username>
```

- For example, `sudo useradd bob`
- By default, sets the home directory to `/home/bob`, populates it with some basic files (copied from `/etc/skel`) and adds a line to `/etc/passwd` such as:

```
bob:x:1002:1002::/home/bob:/bin/bash
```

```
$ sudo userdel <username>
```

- This will leave the `/home/bob` directory intact.
- To remove the home directory while removing the account, one needs to use the `-r` option.

```
$ id
uid=1002(bob) gid=1002(bob) groups=106(fuse),1002(bob)
```

Adding and Removing Users

File Edit View Search Terminal Help

```
c7:/home/coop>sudo useradd -s /bin/csh -m -k/etc/skel -c "Bullwinkle J Moose" bmoose
c7:/home/coop>ls -la /home/bmoose
ls: cannot open directory /home/bmoose: Permission denied
c7:/home/coop>sudo ls -la /home/bmoose
total 28
drwx------  3 bmoose bmoose 4096 Dec 14 09:57 .
drwxr-xr-x. 3 root   root   4096 Dec 14 09:57 ..
-rw-r--r--  1 bmoose bmoose   18 Oct 17 05:05 .bash_logout
-rw-r--r--  1 bmoose bmoose  193 Oct 17 05:05 .bash_profile
-rw-r--r--  1 bmoose bmoose  231 Oct 17 05:05 .bashrc
-rw-r--r--  1 bmoose bmoose  334 Oct  7  2015 .emacs
drwxr-xr-x  4 bmoose bmoose 4096 Jan 26  2014 .mozilla
c7:/home/coop>sudo passwd bmoose
Changing password for user bmoose.
New password:
Retype new password:
passwd: all authentication tokens updated successfully.
c7:/home/coop>su bmoose
Password:
Attempting to create directory /home/bmoose/perl5
[bmoose@c7 coop]$ exit
exit
c7:/home/coop>sudo userdel -r bmoose
c7:/home/coop>ls /home
coop  linux1
c7:/home/coop>
```

Adding and Removing Groups

```
$ sudo groupadd <group_name>

$ sudo groupdel <group_name>

$ groups alice

    alice : alice

$ sudo usermod -a -G anewgroup alice
```

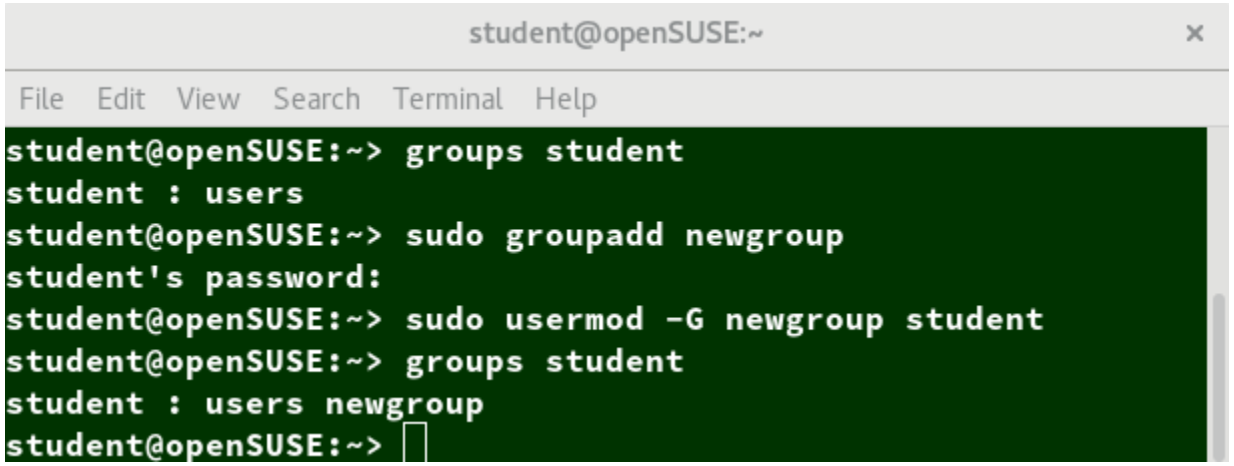
```
$ groups alice

    alice : alice anewgroup

$ sudo usermod -G alice alice

$ groups alice

    alice : alice
```



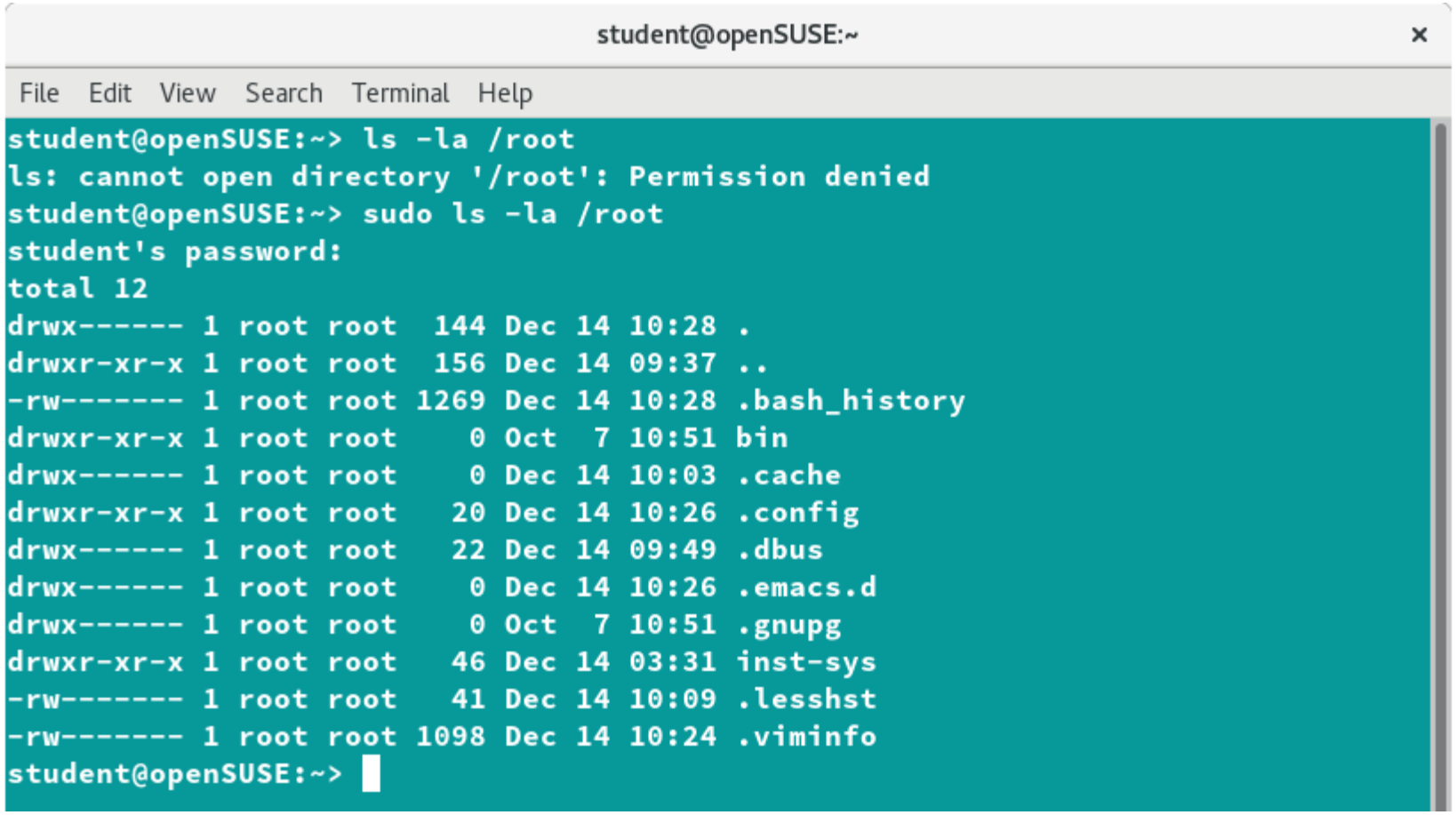
The root Account

- The root account is very powerful and has full access to the system.
- Other OSs often call this the administrator account; in Linux, it is often called the superuser account.
- You must be extremely cautious before granting full root access to a user; it is rarely, if ever, justified. External attacks often consist of tricks used to elevate to the root account.
- You can use `sudo` to assign more limited privileges to user accounts:
  - Only on a temporary basis
  - Only for a specific subset of commands.

`su` and `sudo`

- When assigning elevated privileges, you can use the command `su` (switch or substitute user) to launch a new shell running as another user (you must type the password of the user you are becoming).
  - Most often, this other user is root, and the new shell allows the use of elevated privileges until it is exited.
  - It is almost always a bad (dangerous for both security and stability) practice to use `su` to become root.
  - Resulting errors can include deletion of vital files from the system and security breaches.
- Granting privileges using `sudo` is less dangerous and is preferred.
  - By default, `sudo` must be enabled on a per-user basis.
  - Some distributions (such as Ubuntu) enable it by default for at least one main user or give this as an installation option.
  - `sudo` configuration files are stored in the `/etc/sudoers` file and in the `/etc/sudoers.d/` directory. By default, the `sudoers.d` directory is empty.

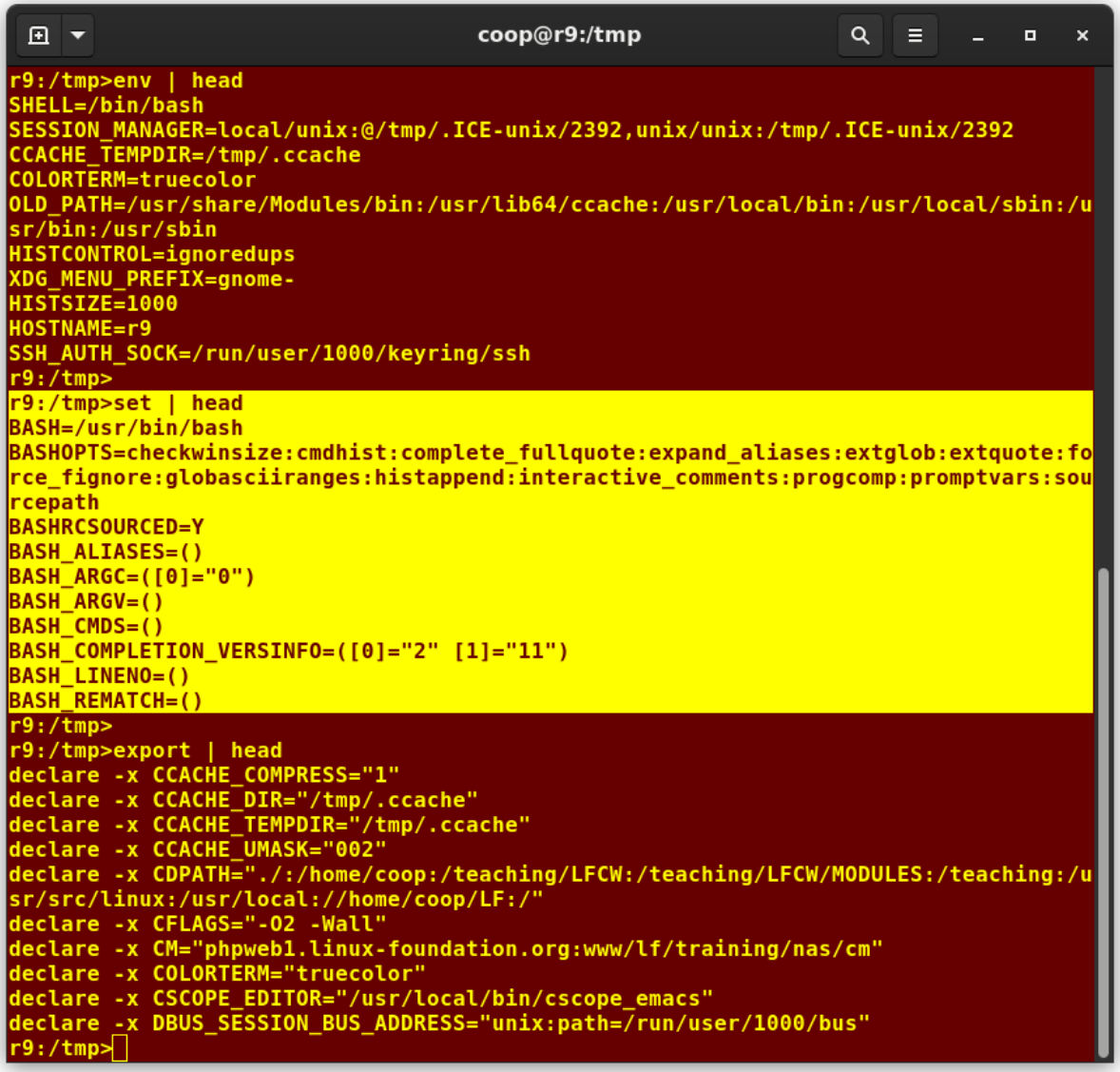
Evaluating Root Account



Environment Variables

- Environment variables are quantities that have specific values which may be utilized by the command shell, such as bash, or other utilities and applications.
- Some environment variables are given preset values by the system (which can usually be overridden), while others are set directly by the user, either at the command line or within startup and other scripts.
- An environment variable is actually just a character string that contains information used by one or more applications.
- There are a number of ways to view the values of currently set environment variables; one can type `set`, `env`, or `export`.
    - Depending on the state of your system, `set` may print out many more lines than the other two methods.

Environment Variables

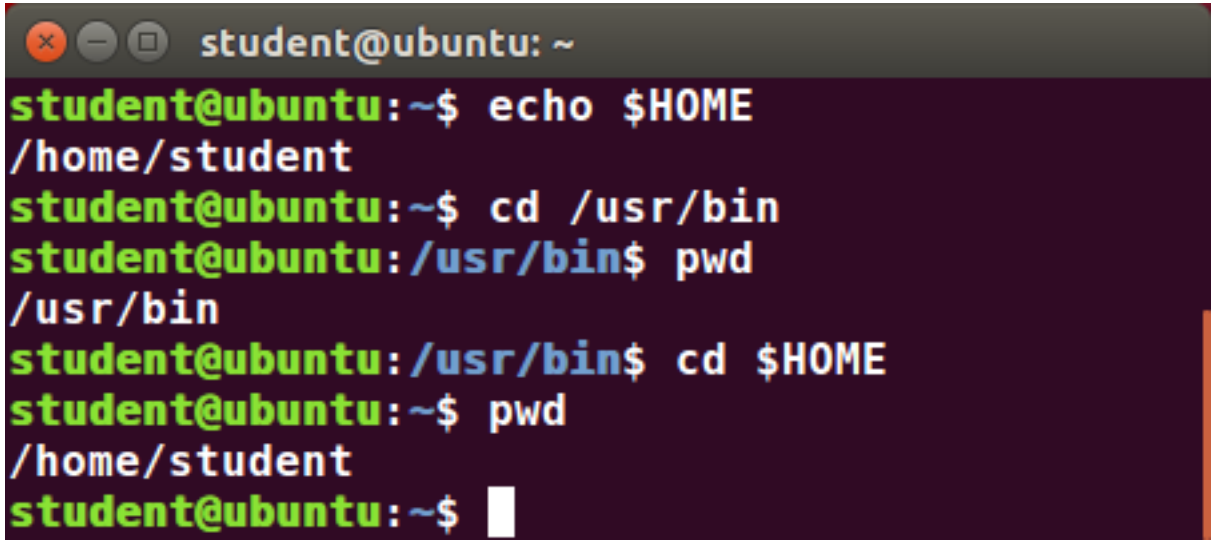


Setting Environment Variables

Task	Command
Show the value of a specific variable	`echo $SHELL`
Export a new variable value	`export VARIABLE=value` (or `VARIABLE=value; export VARIABLE`)
Add a variable permanently	Edit `~/.bashrc` and add the line `export VARIABLE=value` Type `source ~/.bashrc` or just `. ~/.bashrc;` or just start a new shell by typing bash

The HOME Variable

- HOME is an environment variable that represents the home (or login) directory of the user.
- cd without arguments will change the current working directory to the value of HOME.
- Note the tilde character (~) is often used as an abbreviation for $HOME. Thus, cd $HOME and cd ~ are completely equivalent statements.



The `PATH` Variable

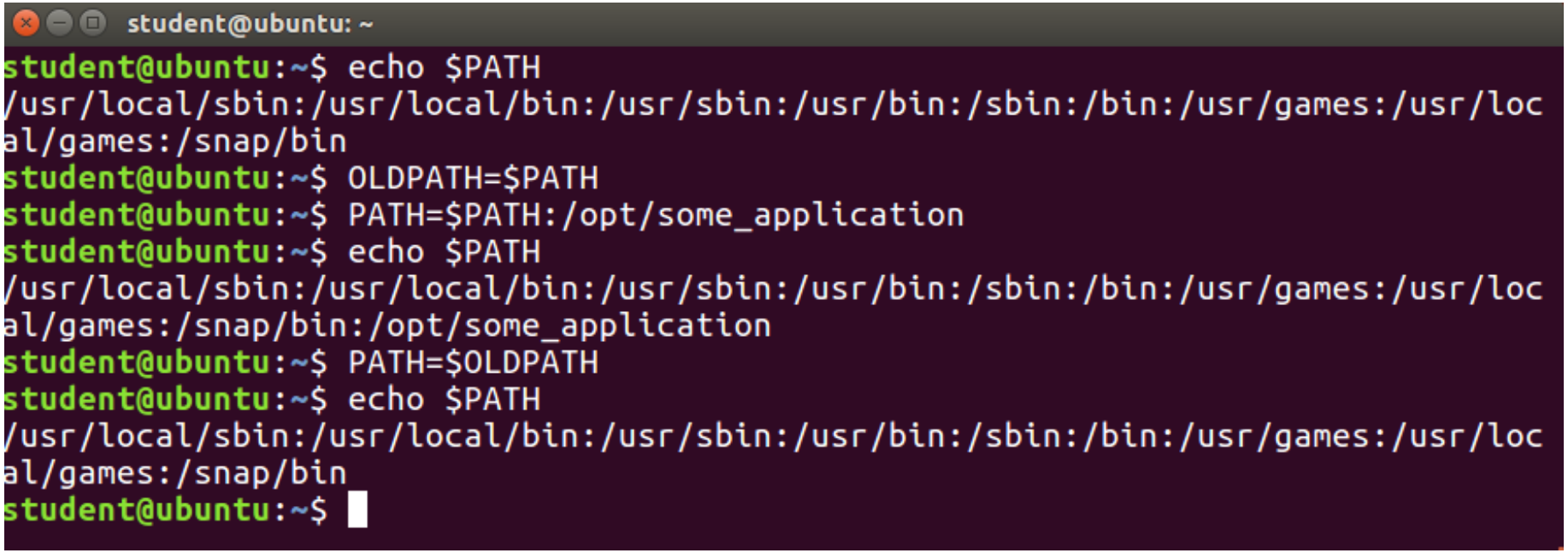
- `PATH` is an ordered list of directories (the path) which is scanned when a command is given to find the appropriate program or script to run.
  - `path1:path2`
  - `:path1:path2`
  - `path1::path2`
- To prefix a private bin directory to your path

```
$ export PATH=$HOME/bin:$PATH

$ echo $PATH

/home/student/bin:/usr/local/bin:/usr/bin:/bin/usr
```

The PATH Variable



The SHELL Variable

- The environment variable SHELL points to the user's default command shell (the program that is handling whatever you type in a command window, usually bash) and contains the full pathname to the shell:

```
$ echo $SHELL

/bin/bash
```

The `PS1` Variable and the Command Line Prompt

- Prompt Statement (`PS`) is used to customize your prompt string in your terminal windows to display the information you want.
- `PS1` is the primary prompt variable which controls what your command line prompt looks like.
  - `\u` - User name
  - `\h` - Host name
  - `\w` - Current working directory
  - `\!` - History number of this command
  - `\d` - Date

```
$ echo $PS1

$

$ export PS1='\u@\h:\w$ '

student@example.com:~$
```

The `PS1` Variable and the Command Line Prompt

- To revert the changes:

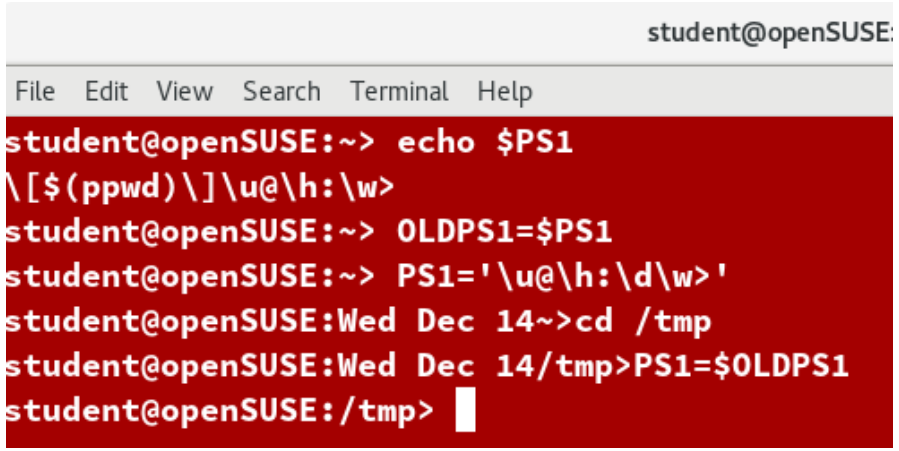
```
student@example.com:~$ export PS1='$ '
$
```

- An even better practice would be to save the old prompt first and then restore

```
$ OLD_PS1=$PS1
```

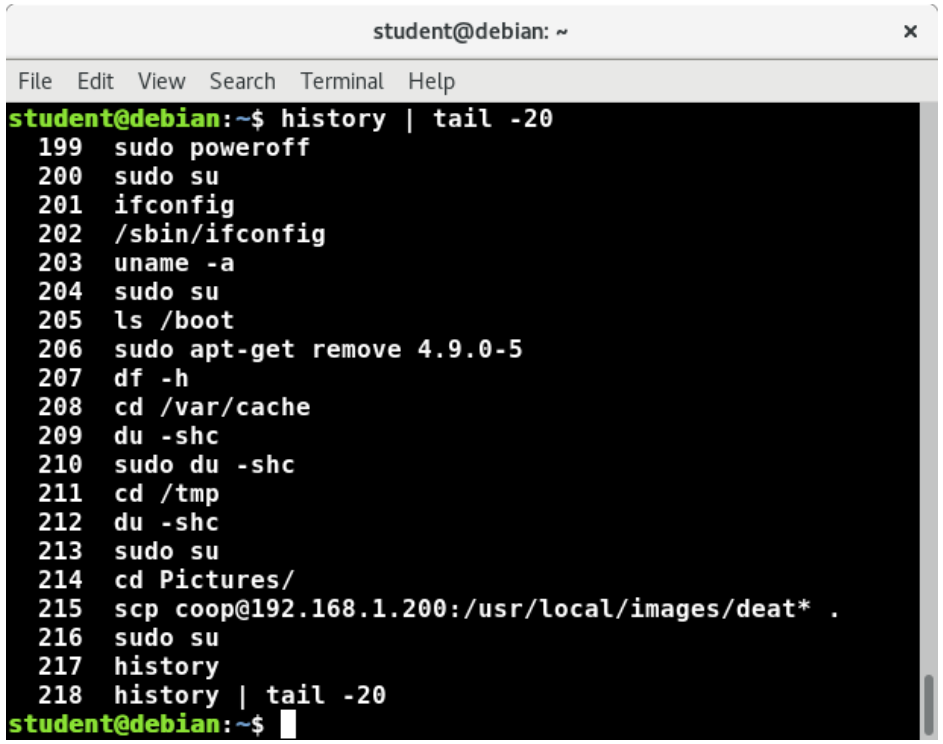
- change the prompt, and eventually change it back with:

```
$ PS1=$OLD_PS1
$
```



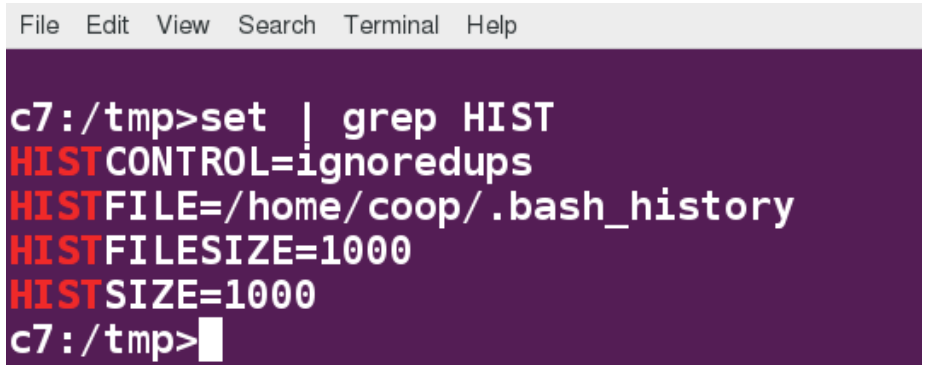
Recalling Previous Commands

- `bash` keeps track of previously entered commands and statements in a history buffer.
- We can recall previously used commands simply by using the Up and Down cursor keys.
- This information is stored in `~/.bash_history`.



Using History Environment Variables

- `HISTFILE` = The location of the history file.
- `HISTFILESIZE` = The maximum number of lines in the history file (default 500).
- `HISTSIZE` = The maximum number of commands in the history file.
- `HISTCONTROL` = How commands are stored.
- `HISTIGNORE` = Which command lines can be unsaved.



Finding and Using Previous Commands

Key	Usage
Up/Down arrow keys	Browse through the list of commands previously executed
`!!` (Pronounced as bang bang)	Execute the previous command
`CTRL-R`	Search previously used commands

```
$ ^R                                          (This all happens on 1 line)

(reverse-i-search)'s': sleep 1000             (Searched for 's'; matched "sleep")

$ sleep 1000                                  (Pressed Enter to execute the searched command)

$
```

Executing Previous Commands

Syntax	Task
`!`	Start a history substitution
`!$`	Refer to the last argument in a line
`!`*`n`*	Refer to the nth command line
`!string`	Refer to the most recent command starting with string

All history substitutions start with `!`. When typing the command:

```
ls -l /bin /etc /var
```

`!$` will refer to `/var`, the last argument to the command.

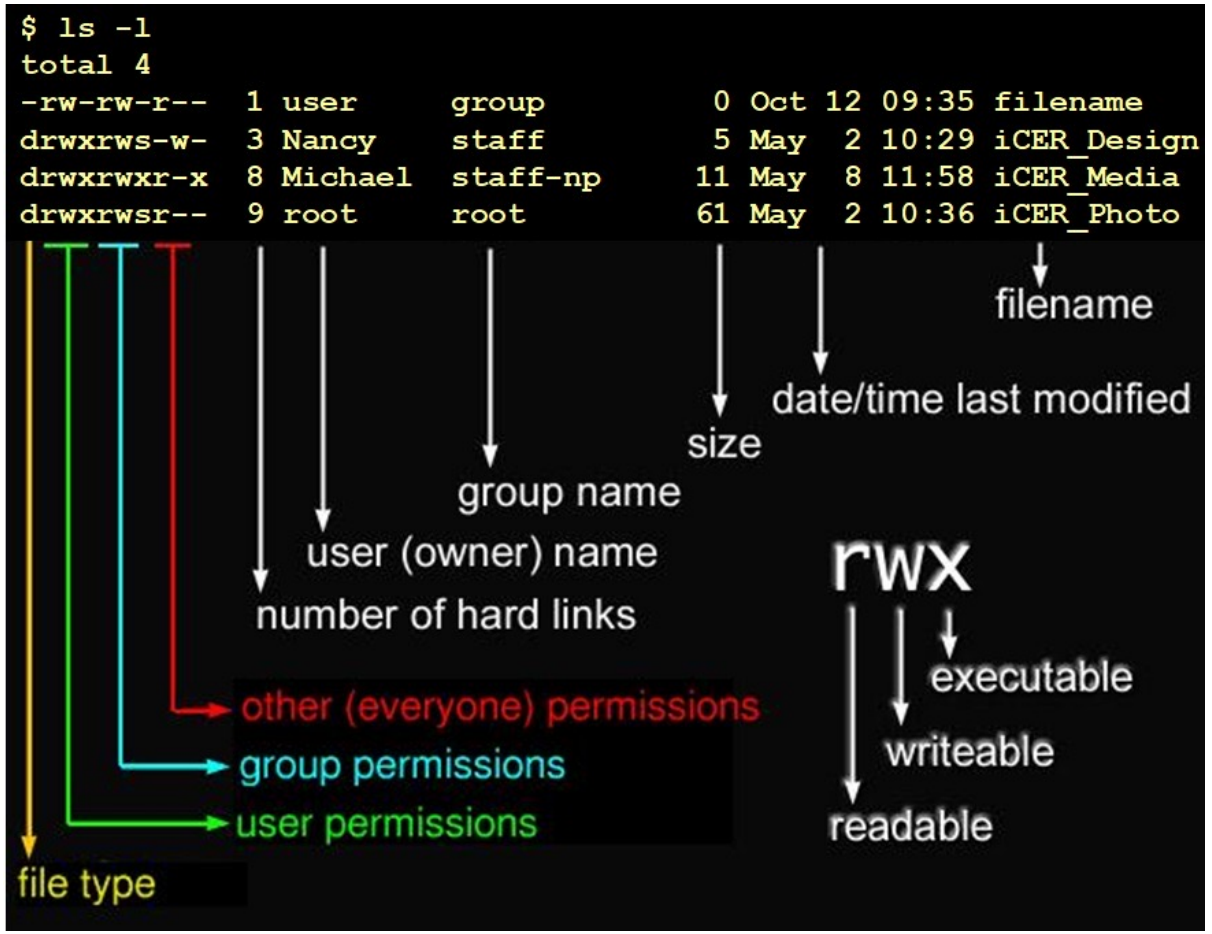
Executing Previous Commands

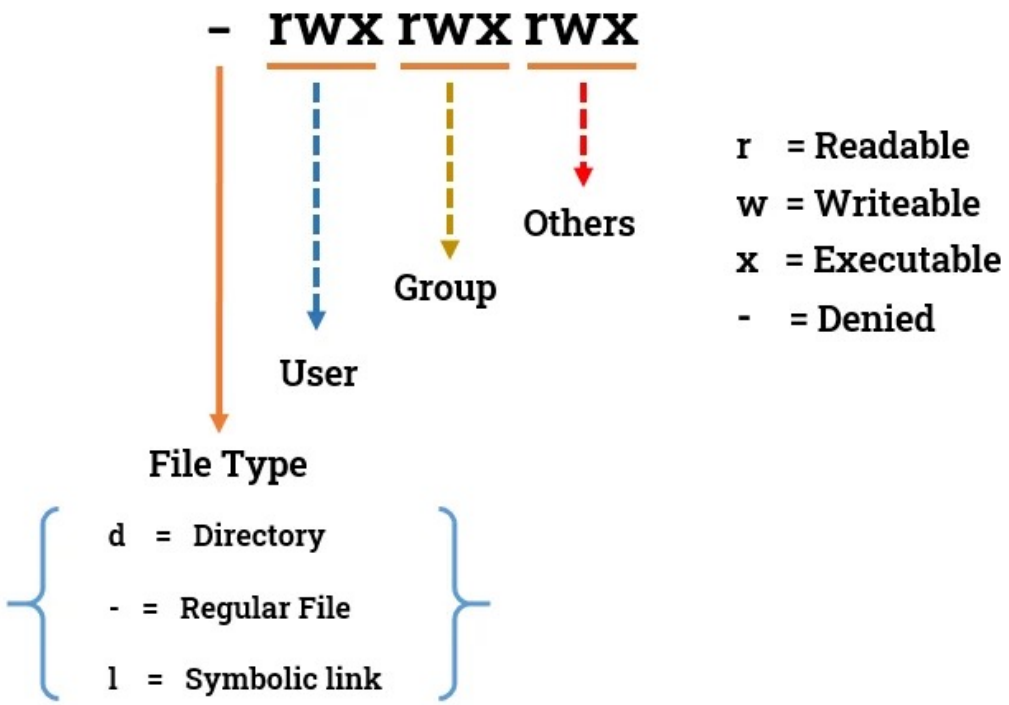
```
$ history
    1.echo $SHELL
    2.echo $HOME
    3.echo $PS1
    4.ls -a
    5.ls -l /etc/ passwd
    6.sleep 1000
    7.history
$ !1                    (Execute command #1 above)
echo $SHELL
/bin/bash
$ !sl                   (Execute the command beginning with "sl")
sleep 1000
$
```

Keyboard Shortcuts

Keyboard Shortcut	Task
CTRL-L	Clears the screen
CTRL-S	Temporarily halt output to the terminal window
CTRL-Q	Resume output to the terminal window
CTRL-D	Exits the current shell
CTRL-Z	Puts the current process into suspended background and back to prompt
CTRL-C	Kills the current process
CTRL-H	Works the same as backspace
CTRL-A	Goes to the beginning of the line
CTRL-W	Deletes the word before the cursor
CTRL-U	Deletes from beginning of line to cursor position
CTRL-E	Goes to the end of the line
Tab	Auto-completes files, directories, and binaries

File Permission

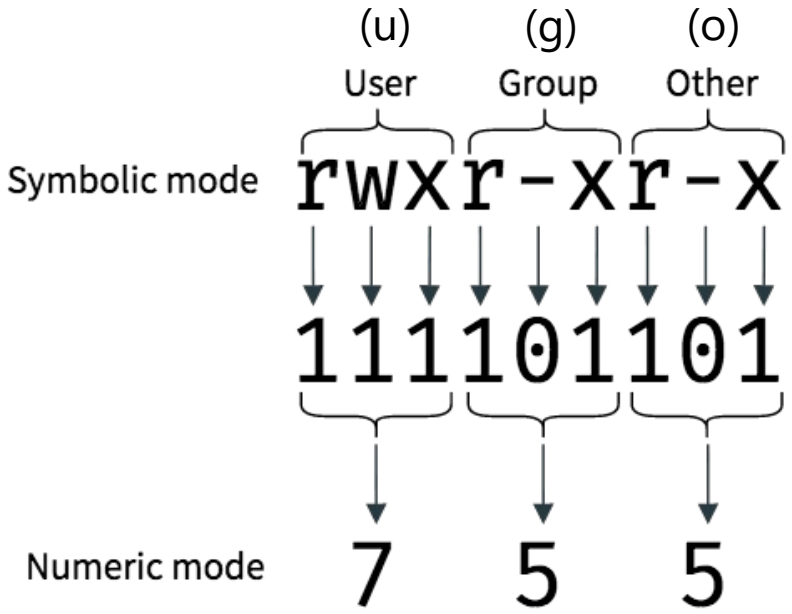




https://docs.icer.msu.edu/Managing_File_Permissions_on_HPCC/
https://linuxopsys.com/linux-file-permissions-explained-co11

File Permission and Ownership

Command	Usage
`chown`	Used to change user ownership of a file or directory
`chgrp`	Used to change group ownership
`chmod`	Used to change the permissions on the file, which can be done separately for owner, group and the rest of the world (often named as other)



	4	2	1	
0	-	-	-	no permissions
1	-	-	x	only execute
2	-	w	-	only write
3	-	w	x	write and execute
4	r	-	-	only read
5	r	-	x	read and execute
6	r	w	-	read and write
7	r	w	x	read, write and execute

Source: https://avabodha.in/linux-file-folder-permissions/

File Permission Example

```
$ ls -l somefile

-rw-rw-r-- 1 student student 1601 Mar 9 15:04 somefile

$ chmod uo+x,g-w somefile

$ ls -l somefile

-rwxr--r-x 1 student student 1601 Mar 9 15:04 somefile

$ chmod 755 somefile

$ ls -l somefile

-rwxr-xr-x 1 student student 1601 Mar 9 15:04 somefile
```

Example of `chown` and `chgrp`

```
File  Edit  View  Search  Terminal  Help
c7:/tmp>touch file1 file2
c7:/tmp>ls -l file?
-rw-rw-r-- 1 coop coop 0 Jan  5 11:48 file1
-rw-rw-r-- 1 coop coop 0 Jan  5 11:48 file2
c7:/tmp>sudo chown root file2
c7:/tmp>ls -l file?
-rw-rw-r-- 1 coop coop 0 Jan  5 11:48 file1
-rw-rw-r-- 1 root coop 0 Jan  5 11:48 file2
c7:/tmp>sudo chown root:root file?
c7:/tmp>ls -l file?
-rw-rw-r-- 1 root root 0 Jan  5 11:48 file1
-rw-rw-r-- 1 root root 0 Jan  5 11:48 file2
c7:/tmp>rm file?
rm: remove write-protected regular empty file ‘file1’? y
rm: cannot remove ‘file1’: Operation not permitted
rm: remove write-protected regular empty file ‘file2’? y
rm: cannot remove ‘file2’: Operation not permitted
c7:/tmp>sudo rm file?
c7:/tmp>
```

```
File  Edit  View  Search  Terminal  Help
c7:/tmp>touch file1
c7:/tmp>ls -l file1
-rw-rw-r-- 1 coop coop 0 Jan  5 12:00 file1
c7:/tmp>sudo chgrp bin file1
c7:/tmp>ls -l file1
-rw-rw-r-- 1 coop bin 0 Jan  5 12:00 file1
c7:/tmp>rm file1
rm: remove regular empty file ‘file1’? y
c7:/tmp>
```